

Internal information

From Reinhard Enste (cpsreen)
Department EE

Date 19/06/2008

Version 3.05

Title **Solar Inverter Communication Protocol**

1. Description:

The protocol is based on an addressing system, in which the system controller (energrid, industrial pc, laptop etc.) is the master and the solar inverters (maximum number of 32 inverters in a RS485 network; only 1 device in a RS232 network) are slaves. The master initiates all communications and the slaves respond only to the command which contains their specific address.

2. Physical layer:

The physical layer uses a uart device in asynchronous communication mode. The outside hardware is RS485 and RS232 compatible.

Hint: Only one of the outside hardware can be used in time. I.e. if the RS485 is used, the RS232 can not be used.

Communication settings:

baudrate (selectable via display): 2400 baud
4800 baud
9600 baud
19200 baud (default value)
38400 baud

number of databits (fixed value): 8
parity (fixed value): none parity
number of stop bits (fixed value): 1

3. Protocol Characters:

The characters in table are used in the communication protocol.

Character	ASCII (hex) Code	Description
'ENQ'	05	Link request (Master only)
'ACK'	06	Slave accepting link (Slave only)
'NAK'	15	Slave not accepting link (Slave only)
'STX'	02	Message start
'ETX'	03	Message end

4. Device Addresses:

For the communication the device addresses 1 to 254 can be selected. The address 255 is used for global calls to all slave devices. The device address is in 8-bit integer format.

5. Protocol

The next tables show the three possible communication protocols.

Packet 1 shows a communication from the master to one or all slave devices.

Packet 2 show the response from a slave device with an acknowledge for correct interpretation of the received data.

Packet 3 show the response from a slave device with a not acknowledge for the received data. The data can be an invalid command, subcommand or the N bytes of data.

Packet 1: Master to slave

Byte #	Databyte	Description
1	'STX'	Start of protocol
2	'ENQ'	Master sending request
3	address	Address of receiving device
4	# of data bytes	Number of data included commands
5	command	Command send to slave
6	sub command	Sub command send to slave
N	data	N bytes of data
N + 1	crc low byte	Low byte of checksum
N + 2	crc high byte	High byte of checksum
N + 3	'ETX'	End of protocol

Packet 2: Slave response with acknowledge to master

Byte #	Databyte	Description
1	'STX'	Start of protocol
2	'ACK'	Slave acknowledging
3	address	Address of slave responding
4	# of data bytes	Number of data included commands
5	command	Repeat command being responded to
6	sub command	Repeat sub command being responded to
N	data	N bytes of data
N + 1	crc low byte	Low byte of checksum
N + 2	crc high byte	High byte of checksum
N + 3	'ETX'	End of protocol

Packet 3: Slave response with no acknowledge to master

Byte #	Databyte	Description
1	'STX'	Start of protocol
2	'NAK'	Slave not acknowledging
3	address	Address of slave responding
4	# of data bytes	Number of data included commands
5	command	Repeat command being responded to
6	sub command	Repeat sub command being responded to
7	crc low byte	Low byte of checksum
8	crc high byte	High byte of checksum
9	'ETX'	End of protocol

Hint 1: The checksum is the result from a CRC16 algorithm not including the framing ('STX' and 'ETX') characters. The CRC16 is calculated by the following polynomial: $X^{16} + X^{15} + X^2 + 1$.

Hint 2: the slave answer with NAK only if the command is invalid.

Hint 3: the slave answer to a valid writing command with the ACK frame and the data value is the internal slave data. If the slave receive a value out of the internal limits it will ignore the received data and set the internal value according to the allowed limits.

6. CRC-16 calculation

The following 'C' code listing correctly implements the CRC-16 calculation intended for use in the solar control network. This CRC-16 implementation is consistent with the most commonly used CRC-16 'standard'

```
unsigned int calc_crc(unsigned char *sop,unsigned char *eop){
    unsigned int crc;
    unsigned char bit_count;
    unsigned char *char_ptr;

    char_ptr = sop;

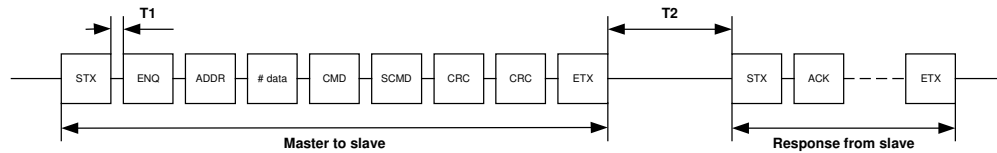
    crc = 0x0000;                //initialize to zero, not all 1's
    do{
        crc^=((*char_ptr)&0x00ff);    //make sure only 8-bits get modified
        bit_count = 0;
        do{
            if(crc&0x0001){          //test before shifting
                crc>>=1;
                crc^=0xA001;        //reflected version of poly:0x8005
            }else{
                crc>>=1;
            }
        }while(bit_count++ < 7);    //for every bit
    }while(char_ptr++ < eop);        //for every byte
    return crc;                    //return 16 bits of crc
}
```

Once the CRC-16 is implemented, testing the algorithm for consistency is easy:

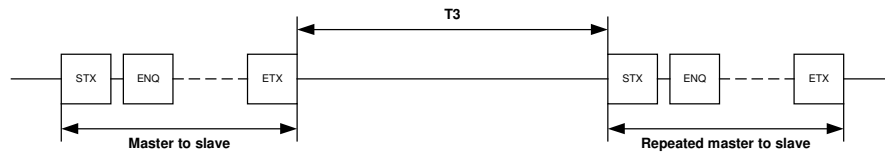
- Input the ASCII string of “123456789” through the calculation. It should return a value of 0xBB3D
- Send the ASCII string back through the calculation, with the two bytes of the CRC appended to the end in *reversed* order: “123456789”,0x3D,0xBB. The returned value should now be 0x0000

7. Timing

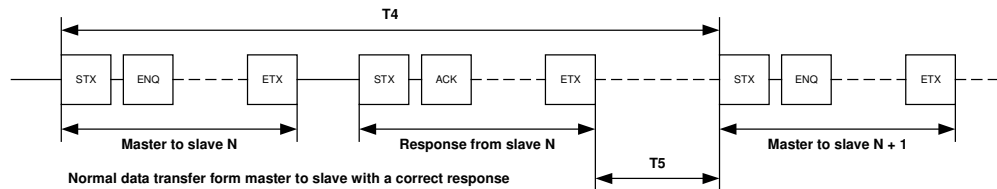
The diagrams show the timing of the communication. The time T1 to T5 differs depending on the baudrate setting. See table for different values.



Normal data transfer form master to slave with a correct response



Data transfer form master to slave with no response from slave



Normal data transfer form master to slave with a correct response

Baudrate	T1	T2	T3	T4	T5
2400	< 12 mS	≥ 5 mS	> 48 mS	> 1920 mS	> 48 mS
4800	< 6 mS	≥ 5 mS	> 24 mS	> 960 mS	> 24 mS
9600	< 3 mS	≥ 5 mS	> 12 mS	> 480 mS	> 12 mS
19200	< 1,5 mS	≥ 5 mS	> 6 mS	> 240 mS	> 6 mS
38400	< 0,75 mS	≥ 5 mS	> 3 mS	> 120 mS	> 3 mS

8. Revision History

- V3.04 - remove gridfit 2200 name from document
- change title into Solar Inverter Communication Protocol
- V3.05 - added hint 2 and hint 3